

Toollery: Scaling LLM Agents to Thousands of Skills and Tools

Anonymous ACL submission

Abstract

As LLM agents are exposed to hundreds to tens of thousands of skills, tools, and API functions, full-library prompting becomes costly, slow, and less reliable: each added candidate increases prompt tokens and latency, while longer candidate lists introduce more distractors for LLM selection. We introduce **Toollery**, a training-free candidate-compression framework for scalable LLM skill/tool selection. Toollery generates user-intent queries from each skill/tool specification and builds a retrieval index that maps real user requests to compact candidate sets before final LLM decision-making. By treating high-level skills and atomic tools as selectable capabilities, Toollery can be applied to both skill libraries and tool registries. We evaluate Toollery on SkillsBench with roughly 79K skills, BFCL-V4 with over 440 atomic tools, and a proprietary 220-tool smart-cockpit dataset with multi-tool requests. Across these settings, Toollery keeps online selection bounded to a compact top- k candidate set, improves recall over ordinary specification retrieval, and improves BFCL-V4 end-to-end tool-selection accuracy compared with full-library prompting. These results support Toollery as a practical framework for large and evolving agent capability libraries: shift semantic matching offline, keep online prompts compact, and let the final LLM decide only among a compact set of retrieved candidates.

1 Introduction

LLM agents increasingly rely on external resources to complete user requests: a resource may be a high-level procedural skill, an MCP service, an atomic tool, or an API function (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024). Although these resources differ in granularity and execution interface, they play the same role before execution: the agent must decide whether each one should be exposed to the final LLM for the current user request.

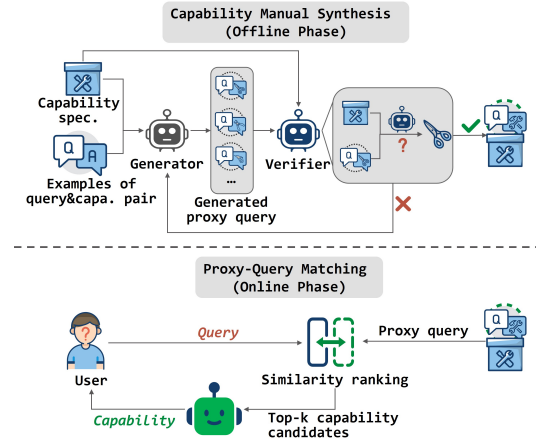


Figure 1: **Overview of Toollery.** Toollery constructs a manual by generating user-intent queries from each capability specification. At inference time, it retrieves a compact candidate set using the manual and passes only the retrieved candidate specifications to the final LLM for decision making.

We therefore treat them as selectable *capabilities* throughout this paper. As capability libraries grow, skill/tool selection becomes a production bottleneck. Industrial reports and our own agent deployments show that loading skill/tool specifications upfront can consume tens of thousands of tokens before any user work begins, while selection accuracy degrades as the library grows (Anthropic, 2026, 2025). In our deployment, after loading more than 200 skills and tools, even a trivial greeting can consume more than 50K input tokens and over 5 seconds of response latency. Scalable LLM skill/tool selection is therefore necessary for controlling token cost, latency, and accuracy as public ecosystems aggregate increasingly large capability libraries (OpenClaw, 2026; Lin et al., 2025; Mo et al., 2025; Xu et al., 2025).

The common practice is full-library prompting: place every skill/tool specification in the prompt and let the LLM choose. This preserves the full decision context, but makes the agent pay the cost

of reviewing every available capability on every request, even when the request needs no tool. As a result, easy or irrelevant requests still incur high prompt cost, higher latency, and more distractors for final LLM decision making. Practical agents therefore need a candidate-compression layer that narrows the action space before LLM inference.

Retrieval is a natural way to implement candidate compression, but ordinary specification retrieval is limited. Existing retrieval-based methods commonly retrieve candidates directly from skill/tool specifications, including names, descriptions, and schemas (Agarwal et al., 2025; Lin et al., 2025; Zheng et al., 2026). This assumes that user requests resemble specifications. In practice, users describe goals, constraints, or partial tasks, while specifications describe capabilities, parameters, and developer-facing usage details. This intent-specification mismatch is especially problematic for heterogeneous libraries whose capabilities are written at different levels of abstraction.

We introduce **Toollery**, a training-free framework that frames scalable LLM skill/tool selection as **intent-level retrieval**. Rather than retrieving over specifications directly, Toollery asks what kinds of user requests should select each capability. It treats high-level skills, MCP services, atomic tools, and API functions as selectable capabilities under the same abstraction. Toollery generates user-intent queries from each capability specification, retrieves a compact top- k candidate set using these intent-augmented representations, and forwards only the selected candidates to the final LLM. This separates library-scale matching from schema-aware final LLM decision making.

Our experiments show that Toollery turns the growing skill/tool-selection bottleneck into a bounded retrieval problem. On SkillsBench, BFCL-V4, and a proprietary smart cockpit dataset, candidate pools range from hundreds of tools to roughly 79K skills, yet Toollery keeps online LLM selection bounded to a compact top- k candidate set. This reduces the number of specifications exposed to the final LLM, keeping token cost and latency controllable as the library grows. At the same time, the retrieved top- k sets achieve higher recall than ordinary specification retrieval, and BFCL-V4 end-to-end experiments show that Toollery improves final tool-selection accuracy compared with full-library prompting.

This paper makes the following contributions:

- We formulate skill/tool selection as intent-level candidate compression, treating high-level skills, MCP services, atomic tools, and API functions as selectable capabilities under the same abstraction.
- We present Toollery, a training-free framework that shifts semantic matching offline by generating user-intent queries from capability specifications and retrieving compact top- k candidate sets before final LLM decision making.
- We evaluate Toollery across large-scale skill routing, public atomic-tool benchmarks, and proprietary vehicle control tool data, showing that bounded top- k selection reduces prompt tokens, latency, and distractor exposure while improving retrieval recall and end-to-end accuracy.

2 Related Work

Prior work improves LLM tool use by training or evaluating models that invoke external tools once relevant tools are available in context or training data (Yao et al., 2022; Schick et al., 2023; Patil et al., 2024; Qin et al., 2023; Li et al., 2023; Patil et al., 2025). Toollery addresses the complementary deployment problem of selecting a compact candidate set from a large skill library before final LLM decision-making.

Recent retrieval and routing methods reduce the action space before tool selection (Agarwal et al., 2025; Lin et al., 2025; Zheng et al., 2026; Gan and Sun, 2025; Paramanayakam et al., 2025). MassTool learns search-based tool retrieval signals, SkillRouter retrieves and reranks skills at scale, and RAG-MCP-style systems retrieve tool specifications to reduce prompt bloat. Toollery is complementary: it trains no router, and expands each searchable capability specification with generated user intents so standard retrievers can match user phrasing to developer-facing specifications.

Query and document expansion have been shown to reduce vocabulary mismatch in retrieval (Gao et al., 2023; Gospodinov et al., 2023). Toollery follows a similar approach but differs in a more agentic direction - we operationalize expansion as a capability-conditioned manual construction layer rather than as generic document augmentation. Toollery does not propose a new retriever; its contribution is a deployment-oriented

candidate-compression pipeline that uses generated intent manuals to bridge user requests and heterogeneous skill/tool specifications before final LLM selection.

3 The Toollery Design

Toollery separates full-library search from final LLM decision-making. Figure 1 shows the overall pipeline: Toollery first generates user-intent queries for each selectable capability, then uses a standard retriever at inference time to retrieve a compact candidate set for the final LLM. Throughout the paper, we use *capability* as the general unit of selection: a capability may be a high-level procedural skill, an MCP service, an atomic tool, or an API function. Toollery can retrieve from either generated user-intent queries alone or skill/tool specifications augmented with those generated queries; the latter preserves both user-facing intent signals and developer-facing specification details.

3.1 Problem Formulation

Let $\mathcal{A} = \{a_1, \dots, a_M\}$ denote the full library of available capabilities, where each capability may be a high-level skill, an MCP service, an atomic tool, or an API function. Each capability a_i has a specification s_i , which may include its name, description, schema, instruction, usage constraints, examples, or documentation. Given a user request x , the goal is to construct a compact candidate set

$$\mathcal{C}_k(x) \subset \mathcal{A}, \quad |\mathcal{C}_k(x)| = k \ll M, \quad (1)$$

that contains the capability or capabilities needed to satisfy the request. The final LLM then receives x and the specifications of only the candidates in $\mathcal{C}_k(x)$, and produces the final selection and any required arguments.

The main difficulty is the mismatch between user requests and specifications. Users describe goals, constraints, or partial tasks, while specifications describe capabilities, parameters, and usage details. Toollery addresses this mismatch by constructing a manual for the full library: a set of generated user-intent queries that describe when each capability should be selected.

3.2 Manual Construction

For each capability a_i , a generator model G produces N user-intent queries from its specification:

$$Q_i = G(s_i; N, \pi), \quad (2)$$

where π prompts the generator to write realistic user requests that would require a_i . The manual for the full library is the union of the generated user-intent queries for all capabilities:

$$\mathcal{M} = \{(q, a_i) : q \in Q_i, a_i \in \mathcal{A}\}. \quad (3)$$

Each manual entry stores a generated user-intent query and the identifier of the capability whose specification produced it.

Toollery optionally applies an offline source-capability recovery filter to improve manual quality. For each generated user-intent query, a verifier is asked to identify the source capability from a small pool containing that capability and randomly sampled distractors. Toollery keeps the query only when the verifier recovers the source capability, filtering out intents that are too broad or non-discriminative. This step uses no evaluation requests or gold labels, so it acts as an offline quality-control step for generated intents rather than a benchmark-specific reranker. Appendix A provides implementation details, Appendix A.1 gives the prompt template, and Appendix A.3 shows a concrete walkthrough.

3.3 Retrieval over the Manual

At inference time, Toollery retrieves candidates using the user request x and the manual $\mathcal{M}$. The retriever is not tied to a specific implementation: it can be instantiated with sparse lexical retrieval such as BM25, dense retrieval, or a RAG-style retriever. In this work, we provide two Toollery retrieval settings. The first setting scores generated user-intent queries in the manual directly, and the second setting scores specifications augmented with the corresponding user-intent queries from the manual.

When retrieval is performed over manual entries, multiple retrieved user-intent queries may point to the same capability. Toollery therefore aggregates query-level evidence into capability-level scores. Let $f(x, q)$ be the retrieval score between user request x and a generated user-intent query q . Toollery ranks each capability by aggregating the strongest matching queries associated with that capability:

$$S(x, a_i) = \text{Agg}\{f(x, q) : (q, a_i) \in \mathcal{M}\}. \quad (4)$$

The candidate set is then

$$\mathcal{C}_k(x) = \text{TopK}_{a_i \in \mathcal{A}} S(x, a_i). \quad (5)$$

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	309.4	0.553	0.571	0.726	0.625	0.512	0.413
Raw specification	RAGAnything	38210.6	0.600	0.568	0.712	0.663	0.500	0.393
Raw specification	SkillRouter [†]	495.8	0.740	0.704	0.814	0.791	0.670	0.520
Toollery (manual only)	BM25	218.9	0.840	0.880	0.951	0.881	0.846	0.760
Toollery (manual only)	RAGAnything	21505.9	0.900	0.922	0.953	0.925	0.882	0.827
Toollery (manual + specification)	BM25	623.0	0.920	0.942	0.968	0.942	0.880	0.853
Toollery (manual + specification)	RAGAnything	41070.5	0.933	0.900	0.950	0.943	0.843	0.827

Table 1: SkillsBench retrieval over roughly 79K skills, averaged over the easy and hard splits. Raw specification retrieves from original skill specifications only. Full-library prompting at this scale would require roughly 43.1M input tokens per request; Toollery top-10 prompting requires roughly 27.7K tokens, over three orders of magnitude smaller. [†] denotes the SkillRouter reference using SR-Emb-0.6B $\times$ SR-Rank-0.6B.

In our implementation, Agg averages the strongest retrieved query scores for each capability and adds a small evidence-count bonus; exact aggregation parameters are listed in Appendix A.

3.4 Candidate Compression and Final Selection

For each selected capability, Toollery keeps only the specification fields required for the final LLM decision. The final LLM receives the original user request and the compact specifications for $\mathcal{C}_k(x)$, rather than the full library or the manual; retrieval narrows the action space, while the LLM resolves residual ambiguity, selects the final capability, and fills arguments.

This design changes the online cost from full-library prompting to subset prompting. Let M be the number of capabilities and L_s the average compact-specification length. Full-library prompting sends approximately $O(M \cdot L_s)$ specification tokens, so prompt cost grows linearly with the library. Toollery instead sends only the retrieved top- k specifications, approximately $O(k \cdot L_s)$ tokens, where $k \ll M$ is a deployment-time candidate budget. Because k can remain fixed as M grows, the final LLM sees a bounded candidate set while the manual and retrieval index absorb library growth offline.

4 Experiments

We evaluate Toollery as a candidate compression layer for scalable capability selection. The experiments test whether Toollery preserves the needed capabilities in compact top- k candidate sets and whether this compression improves end-to-end LLM selection under candidate-pool growth.

4.1 Experimental Setup

Benchmarks. We evaluate Toollery on two public benchmarks and one proprietary industrial dataset. The first public benchmark is SkillsBench

(Li et al., 2026; Zheng et al., 2026), which contains 75 evaluation queries: 24 single-skill queries and 51 multi-skill queries. Each user request is evaluated against an ~ 79 K-skill pool. The second public benchmark is BFCL-V4 (Patil et al., 2024). We use its non-live and live multiple-tool instances and evaluate each request against the complete candidate pool, treating each tool or API function as an atomic capability. Finally, we evaluate on a proprietary smart cockpit dataset with 3,396 real user requests over a 220-tool production pool. Half of the requests contain two gold tools, and the other half contain three gold tools.

Settings and baselines. We compare retrieval settings that differ in what information is available to the retriever. *Raw specification* retrieves only from the original skill/tool specifications and does not use Toollery’s manual. Toollery first constructs a manual for the full library, where the manual contains generated user-intent queries for each capability. We evaluate two ways to retrieve with this manual: *Toollery (manual only)* retrieves from the manual alone, while *Toollery (manual + specification)* retrieves from specifications augmented with the corresponding user-intent queries in the manual. Both are Toollery settings because both use the generated manual; they differ only in whether the retriever also sees the original specification text. Within these settings, we instantiate retrieval with BM25 or RAGAnything.

Metrics and cost. For retrieval, we report Hit@1, Recall@10/50, MRR@10, nDCG@10, FullCoverage@10, and Avg ms/query for warmed online retrieval latency. FullCoverage@10 requires all gold capabilities for multi-capability requests to appear in the top 10; for the proprietary dataset we also report Recall@20. For end-to-end experiments, we report final selection accuracy after the final LLM chooses from the retrieved subset.

Offline manual construction cost is reported

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	3.6	0.643	0.931	0.976	0.744	0.790	0.931
Raw specification	RAGAnything	152.5	0.737	0.976	0.995	0.831	0.867	0.976
Toollery (manual only)	BM25	0.3	0.468	0.822	0.934	0.584	0.642	0.822
Toollery (manual only)	RAGAnything	81.8	0.543	0.855	0.929	0.653	0.702	0.855
Toollery (manual + specification)	BM25	5.0	0.731	0.948	0.986	0.811	0.845	0.948
Toollery (manual + specification)	RAGAnything	139.7	0.776	0.980	0.997	0.856	0.887	0.980

Table 2: BFCL-V4 live multiple retrieval over the complete 457-tool pool.

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	14.4	0.780	0.970	0.995	0.854	0.883	0.970
Raw specification	RAGAnything	524.6	0.825	1.000	1.000	0.895	0.922	1.000
Toollery (manual only)	BM25	0.2	0.555	0.885	0.990	0.668	0.721	0.885
Toollery (manual only)	RAGAnything	77.9	0.605	0.925	0.985	0.722	0.772	0.925
Toollery (manual + specification)	BM25	25.9	0.815	1.000	1.000	0.887	0.915	1.000
Toollery (manual + specification)	RAGAnything	657.3	0.845	1.000	1.000	0.904	0.928	1.000

Table 3: BFCL-V4 non-live multiple retrieval over the complete 443-tool pool.

Dataset	Selection Setting	# Candidates	Input Tokens	Latency	Accuracy
live	Full-library prompt	457	84.1K	11.8s	0.770
live	Toollery	10	2.1K	1.6s	0.831
non-live	Full-library prompt	443	57.0K	8.4s	0.703
non-live	Toollery	10	1.3K	1.5s	0.814

Table 4: BFCL-V4 end-to-end selection with DeepSeek V3.2 under full-library prompting and Toollery top-10 candidate compression.

in Appendix A. Using official DeepSeek-V4-Pro prices as of June 9, 2026,¹ construction costs approximately \$0.22 per 100 skills on average, including generated-query construction and source-capability recovery filtering. This offline cost is excluded from the online retrieval latency numbers in the main tables.

4.2 Skill Retrieval on SkillsBench

SkillsBench tests Toollery at a scale where full-library prompting is infeasible. Prompting over the full ~79K-skill library would require roughly 43M input tokens per request, while Toollery top-10 prompting requires about 27.7K tokens. The question is therefore whether retrieval can compress this full library into a small candidate set without losing the needed capabilities.

Table 1 shows that Toollery substantially improves large-library retrieval. With BM25, Toollery (manual + specification) improves Hit@1 from 0.553 to 0.920, Recall@10 from 0.571 to 0.942, and FullCoverage@10 from 0.413 to 0.853 over raw specification retrieval. The table’s latency column reports retrieval time only; with BM25, Toollery performs this ~79K-skill compression in 623.0 ms/query. Toollery (manual only) is also strong, indicating that the manual itself provides useful user-facing retrieval targets, while adding specifications further improves most metrics.

¹https://api-docs.deepseek.com/quick_start/pricing, accessed June 9, 2026.

4.3 Atomic-Tool Retrieval and End-to-End Selection on BFCL-V4

BFCL-V4 tests whether the same manual-based design transfers from high-level skills to atomic tools and API functions. Unlike SkillsBench, where full-library prompting is infeasible at the full library scale, BFCL-V4 is small enough to run full-library prompting, enabling a direct end-to-end comparison against Toollery. Tables 2 and 3 evaluate retrieval over the complete BFCL-live 457-tool pool and BFCL non-live 443-tool pool.

Toollery remains effective in this atomic-tool setting. With RAGAnything, Toollery (manual + specification) reaches 0.776 Hit@1 and 0.980 Recall@10 on BFCL-live, and 0.845 Hit@1 and 1.000 Recall@10 on BFCL non-live. The manual-only setting slightly underperforms the raw specification retrieval because the atomic tool specs are already short and schema-structured, but the manual still improves top-*k* recall by adding user-intent signals; adding specifications back recovers and exceeds the baseline.

Table 4 shows that this retrieval gain translates into a better end-to-end serving tradeoff with DeepSeek V3.2 as the final selector. On BFCL-live, Toollery reduces input tokens from 84.1K to 2.1K and latency from 11.8s to 1.6s, while improving accuracy from 0.770 to 0.831. On BFCL non-live, it reduces input tokens from 57.0K to 1.3K and latency from 8.4s to 1.5s, while improving accuracy from 0.703 to 0.814. These results show that candidate compression reduces both prompt cost and distractor exposure. Additional end-to-end results across final selectors, decision interfaces, and candidate-pool sizes are reported in Appendix B.

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@20	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	6.4	0.430	0.545	0.696	0.867	0.576	0.446	0.224
Raw specification	RAGAnything	144.5	0.408	0.582	0.718	0.871	0.588	0.464	0.229
Toollery (manual only)	BM25	0.7	0.481	0.670	0.784	0.918	0.648	0.544	0.358
Toollery (manual only)	RAGAnything	221.6	0.453	0.666	0.781	0.902	0.633	0.538	0.357
Toollery (manual + specification)	BM25	69.7	0.484	0.684	0.803	0.926	0.654	0.553	0.381
Toollery (manual + specification)	RAGAnything	253.4	0.403	0.630	0.758	0.893	0.591	0.500	0.316

Table 5: Proprietary smart cockpit retrieval over the complete 220-tool pool. Results are averaged over 3,396 real company-internal requests with two or three gold tools. The proprietary dataset is used only for deployment-facing evaluation and is not released.

4.4 Atomic-Tool Retrieval on Proprietary Dataset

The proprietary smart cockpit dataset evaluates Toollery in a production-style multi-intent setting, where each real user request maps to two or three gold tools from a 220-tool pool. Full-library prompting over this pool would require roughly 35K input tokens per request; Toollery reduces this to about 1.9K tokens with top-10 candidates and about 9.1K tokens with top-50 candidates.

Table 5 shows that Toollery improves candidate-set coverage for real multi-intent requests. Compared with raw BM25 specification retrieval, Toollery improves Recall@10 from 0.545 to 0.684 and Recall@50 from 0.867 to 0.926, and FullCoverage@10 reaches 0.381. These gains are important because multi-intent requests require the compressed candidate set to preserve more than one correct capability. For deployments where completeness is critical, increasing the candidate budget to $k=20$ raises recall to 0.803 at 3.8K tokens, which is still well under the 35K full-library cost. Therefore, Toollery substantially reduces the prompt size of a production capability library, while exposing a clear cost-coverage tradeoff: top-10 retrieval improves over raw specification retrieval, but higher candidate budgets such as top-20 or top-50 are more appropriate when complete multi-intent coverage is required.

5 Discussion and Conclusion

The experiments show that full-library prompting quickly becomes a poor serving strategy as capability libraries grow. At SkillsBench scale, it is infeasible: prompting over the full ~79K-skill library would require roughly 43.1M input tokens per request. Even at smaller scales, the cost remains substantial: BFCL-V4 requires 57.0K–84.1K input tokens and 8.4–11.8 seconds of latency over 443–457 tools, while the 220-tool smart cockpit pool still requires roughly 35.2K input tokens. The deployment pressure is therefore structural: the final LLM pays for every available capability on every

request, including trivial or irrelevant ones.

Toollery addresses this bottleneck by moving semantic matching out of the final prompt and into offline manual construction plus online retrieval. Across all three settings, Toollery keeps final selection bounded to a compact top- k candidate set: SkillsBench drops from roughly 43.1M full-library tokens to 27.7K top-10 tokens; BFCL-V4 drops from 57.0K–84.1K to 1.3K–2.1K tokens; and the proprietary dataset drops from 35.2K to 1.9K top-10 tokens. On BFCL-V4, these reductions also translate into latency drops from 8.4–11.8s to 1.5–1.6s.

The accuracy results show that Toollery does not buy efficiency at the expense of selection quality. Manual-based retrieval improves over raw specification retrieval across SkillsBench, BFCL-V4, and the proprietary multi-intent dataset, and improves candidate-set coverage for multi-capability requests. In BFCL-V4 end-to-end selection, Toollery also improves final tool-selection accuracy over full-library prompting, suggesting that candidate compression reduces distractors as well as prompt cost and latency.

Taken together, these results suggest that scalable capability selection should be treated as candidate compression rather than prompting over an ever-growing full library. Toollery instantiates this principle by constructing a manual from capability specifications offline, retrieving a compact candidate set online, and allowing the final LLM to perform schema-aware decision-making only on the retrieved specifications. The main deployment tradeoff is offline maintenance: the manual and retrieval index must be updated as capabilities are added or changed. This cost is amortized across future requests, while the savings in final-prompt tokens and latency occur on every online request. Overall, Toollery turns large-library capability selection from unbounded full-library prompting into a practical retrieval-and-selection pipeline.

6 Limitations

Toollery depends on the coverage and precision of the manual. If the generated user-intent queries fail to cover realistic ways users ask for a capability, retrieval recall is bounded regardless of the retrieval backend. Conversely, overly broad user-intent queries can introduce noisy matches and increase distractor exposure in the retrieved candidate set. Source-capability recovery filtering reduces this risk by removing queries that do not recover their source capability under distractor competition, but it does not provide a formal correctness guarantee. As shown in Appendix C, remaining failures often involve functionally similar capabilities or mismatches between broad retrieved capabilities and finer-grained benchmark gold labels; exact recall may therefore under-credit plausible alternatives while still exposing real disambiguation challenges.

Toollery also introduces an offline maintenance cost. When capabilities are added, removed, or modified, the corresponding manual entries and retrieval index must be updated. This cost is amortized across future requests and separated from online retrieval latency, but it may matter for rapidly changing capability libraries. Our experiments cover large public skill/tool pools and one proprietary smart cockpit deployment setting; broader validation across more domains, dynamic capability updates, and interactive multi-step agent workflows remains future work.

References

- Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. Toolrm: Outcome reward models for tool-calling large language models. *arXiv preprint arXiv:2509.11963*.
- Anthropic. 2025. Introducing Advanced Tool Use on the Claude Developer Platform. <https://www.anthropic.com/engineering/advanced-tool-use>. Published: 2025-11-24; accessed: 2026-06-09.
- Anthropic. 2026. Tool Search Tool. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool>. Accessed: 2026-06-09.
- Tiantian Gan and Qiyao Sun. 2025. Rag-mcp: Mitigating prompt bloat in llm tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777.
- Mitko Gospodinov, Sean MacAvaney, and Craig Macdonald. 2023. Doc2query–: when less is more. In *European Conference on Information Retrieval*, pages 414–422. Springer.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. *API-bank: A comprehensive benchmark for tool-augmented LLMs*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore. Association for Computational Linguistics.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiansun Sun, and 1 others. 2026. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- Jianghao Lin, Xinyuan Wang, Xinyi Dai, Menghui Zhu, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Masstool: A multi-task search-based tool retrieval framework for large language models. *arXiv preprint arXiv:2507.00487*.
- Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.
- OpenClaw. 2026. ClawHub: Skill and Plugin Registry for OpenClaw. <https://github.com/openclaw/clawhub>. Accessed: 2026-06-09.
- Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2025. Less is more: Optimizing function calling for llm execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.
- Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and Zhijie Deng. 2025. Tps-bench: Evaluating ai agents’ tool planning & scheduling abilities in compounding tasks. *arXiv preprint arXiv:2511.01527*.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.
- Hanjie Zheng, Chenglin Li, Yibing Liu, Jiazhan Feng, Wenhao Wu, Jiabao Guo, Yicheng Peng, Tianhang Li, Qian Zhang, Chi Zhang, and 1 others. 2026. Skill-router: Retrieve-and-rerank skill selection for llm agents at scale. *arXiv preprint arXiv:2603.22455*.

A Reproducibility and Deployment Details

Table 6 summarizes the main configuration choices used in the reported runs. We separate offline manual construction from online retrieval and final selection: manual construction includes user-intent query generation and optional source-capability recovery filtering, while online latency reports only serving-time retrieval or final-selection latency with warmed indexes and caches. We also plan to provide plugin adapters for popular agent runtimes, including OpenClaw, Hermes Agent, Claude Code, and Codex, so that Toollery can be used as a candidate-compression layer in existing agent systems.

A.1 Manual Construction Prompt

The following prompt template is used to generate user-intent queries for manual construction in the SkillsBench and BFCL-V4 runs. Each capability is serialized by its specification, including available fields such as name, description, category, schema, and usage instructions. The generated query fields are stored as manual entries associated with the source capability.

```
Generate realistic user-intent queries for
multiple skills or tools.
For each skill or tool, write things real
users might ask when they need that exact
skill or tool.
Use practical task context, constraints, and
natural wording.
Avoid copying skill text verbatim.
Return only one valid JSON object. The keys
must be the exact skill names.
Each value must be a JSON array of objects.
Each object must contain query, scenario_type,
generation_notes.
Return exactly {N} objects per skill.
```

The returned query fields become manual entries associated with the source capability, while `scenario_type` and `generation_notes` are retained for auditability but are not used by the retriever.

A.2 Source-Capability Recovery Filtering

For each generated user-intent query $q \in Q_i$, Toollery optionally constructs a verification pool $\mathcal{P}_{i,q} = \{a_i\} \cup \mathcal{D}_{i,q}$, where a_i is the source capability whose specification produced q and $\mathcal{D}_{i,q}$ contains eight randomly sampled distractor capabilities from the same full library. A verifier LLM receives q and the specifications of all capabilities in $\mathcal{P}_{i,q}$, then selects the best matching capability or

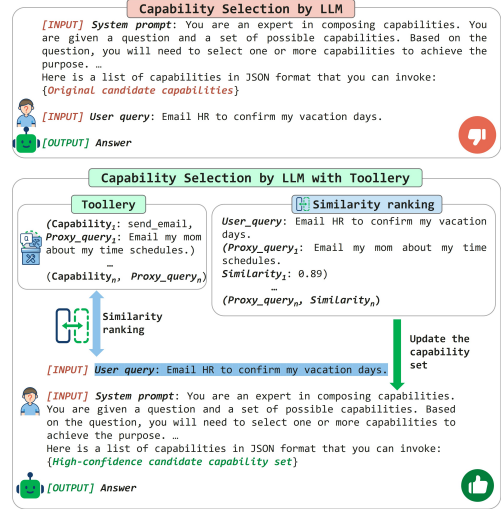


Figure 2: **Toollery pipeline walkthrough.** Toollery converts capability specifications into user-intent queries during offline manual construction. During inference, the user request retrieves matching manual entries and corresponding capability specifications, which are aggregated into a compact candidate set for final LLM decision making.

abstains. Toollery keeps q only when the verifier recovers the source capability a_i .

This filter removes user-intent queries that are too broad or non-discriminative before indexing. Because filtering is performed during offline manual construction, its cost is amortized across future online requests together with query generation.

A.3 Pipeline Walkthrough Example

Figure 2 illustrates one concrete Toollery workflow. Starting from a capability specification, Toollery first generates user-intent queries that describe realistic requests for which the capability should be selected. These generated queries are stored in the manual and remain linked to the source capability. At inference time, a real user request is matched against the manual and, optionally, the original capability specifications. Retrieved manual entries are aggregated back to capability-level scores, producing a compact top- k candidate set. The final LLM then receives only the user request and the retrieved candidate specifications, rather than the full library, and performs the final schema-aware decision. This example highlights the main serving-time change: the final LLM no longer pays the token, latency, and distractor cost of the full library, but only reasons over a bounded set of retrieved candidate specifications.

Item	Value used in reported runs
SkillsBench pools	78,361 easy skills; 79,141 hard skills; 75 requests per split
BFCL-live end-to-end pool	457 tools; 1,053 live multiple requests; final column uses the complete pool
BFCL-non-live end-to-end pool	443 tools; 200 multiple requests; final column uses the complete pool
Manual construction	DeepSeek-V4-Pro through an OpenAI-compatible API; 3 user-intent queries per capability; seed 31; 4 workers; batch size 4
Source-capability recovery verifier	asks DeepSeek-V4-Pro to choose the matching capability id from the source capability plus distractors, or NONE if no candidate fits
Distractor construction	verifier pool uses the source capability plus 8 randomly sampled non-gold distractors; scaled BFCL candidate pools sample the gold tool plus non-gold tools without replacement
BM25 configuration	local tokenizer in <code>toollery.text</code> ; Okapi BM25 with $k_1 = 1.5, b = 0.75$; Toollery (manual + specification) concatenates name, category, description, schema, and user-intent queries from the manual
RAGAnything configuration	text-only RAGAnything/LightRAG cache with one compact chunk per capability; retrieval queries LightRAG chunk vectors and recovers capability ids from returned context order
Retrieval embedding model	Qwen/Qwen3-Embedding-0.6B for neural retrieval and latency replay
Aggregation parameters	$\text{top-}c = 3$ averaged manual-query scores with evidence bonus $b = 5, \lambda = 0.05$
Candidate budgets	top-50 for retrieval-quality tables; top-10 for the main BFCL-V4 end-to-end cost comparison; appendix end-to-end table columns denote the candidate-pool size before Toollery compression
Latency protocol	warmed indexes/caches; reports online retrieval or final-selection latency only; excludes offline manual construction, verification, embedding, and index construction
Offline cost	SkillsBench easy and hard splits; DeepSeek-V4-Pro generator and verifier; 3 user-intent queries per capability; source capability plus 8 random distractors for verification; cost computed with cache-miss input pricing
Release plan	Upon acceptance, we will release the Toollery code, prompts, normalized manual artifacts, run configurations, metric scripts, and plugin adapters publicly.

Table 6: Reproducibility details for the main retrieval and efficiency results. Secrets and local API endpoints are omitted; paths in the released run configurations identify the exact output artifacts.

A.4 Offline Manual Construction Cost

We report offline manual construction cost separately from online latency because this cost is paid before deployment and amortized across future requests. On SkillsBench, we use DeepSeek-V4-Pro as both the generator and the source-capability recovery verifier. Each capability receives three generated user-intent queries, and each non-empty query is verified against its source capability plus eight randomly sampled distractors.

Table 7 reports the measured construction cost for the SkillsBench easy and hard pools. Under official DeepSeek-V4-Pro cache-miss input and output pricing,² offline manual construction costs approximately \$0.22 per 100 skills on average. These values describe offline construction only; all online latency results in the main tables use warmed indexes and exclude generation, verification, embedding, and index construction.

B Additional End-to-End Results

Tables 8 and 9 provide additional BFCL-V4 end-to-end selection results beyond the main DeepSeek

Setting	Skills	Cand.	Ret.	Tok.	Cost
SkillsBench easy	78,361	235K	228K	361.8M	\$168.70
SkillsBench hard	79,141	237K	230K	365.4M	\$170.38

Table 7: Offline construction cost for generated-query construction and verifier filtering. The measured rows use the SkillsBench easy and hard splits with three generated queries per skill. Cost uses official DeepSeek-V4-Pro pricing: \$0.435/M cache-miss input tokens and \$0.87/M output tokens; cache-hit discounts are not applied.

V3.2 comparison in Table 4. We vary the final selector, the decision interface, and the candidate-pool size before Toollery compression. The BFCL-V4 live multiple split contains 1,053 requests and 457 unique tools, while the non-live multiple split contains 200 requests and 443 unique tools; therefore, the final columns correspond to the complete candidate pools for the two splits.

These results test whether candidate compression remains useful beyond a single final LLM. Across prompt-based and function-calling settings, Toollery is most helpful when the candidate pool is large, where full-library prompting exposes the final LLM to more tool specifications and distractors. The trend supports the main result: Toollery improves the serving tradeoff by retrieving a compact

²https://api-docs.deepseek.com/quick_start/pricing/, accessed June 9, 2026.

Table 8: BFCL-V4 live multiple results

Model	≤ 10	20	30	40	50	100	457
Qwen3-4B (Prompt)	85.50	86.00	88.00	82.50	78.00	69.50	24.88
Qwen3-4B (Prompt) + Toollery	87.00 $\uparrow$	84.50	87.00	86.50 $\uparrow$	82.50 $\uparrow$	81.00 $\uparrow$	83.75 $\uparrow$
Qwen3-8B (Prompt)	97.00	93.50	92.00	84.00	83.00	84.50	38.50
Qwen3-8B (Prompt) + Toollery	95.50	98.50 $\uparrow$	98.50 $\uparrow$	92.50 $\uparrow$	94.50	86.50 $\uparrow$	79.38 $\uparrow$
DeepSeek V3.2 (Prompt)	94.50	80.00	84.00	89.50	79.50	63.00	77.00
DeepSeek V3.2 (Prompt) + Toollery	98.00 $\uparrow$	78.50	94.50 $\uparrow$	99.50 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	83.13 $\uparrow$
GPT-5-mini (Prompt)	83.00	93.00	90.00	87.00	88.00	88.00	79.75
GPT-5-mini (Prompt) + Toollery	83.50 $\uparrow$	94.00 $\uparrow$	89.50	95.00 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	84.50 $\uparrow$
GPT-5-mini (FC)	67.00	62.00	63.50	62.00	61.00	63.50	57.00
GPT-5-mini (FC) + Toollery	74.50 $\uparrow$	67.50 $\uparrow$	69.00 $\uparrow$	69.50 $\uparrow$	67.00 $\uparrow$	67.50 $\uparrow$	66.00 $\uparrow$
Qwen3-8B (FC)	93.50	94.50	92.50	83.00	83.50	83.00	60.00
Qwen3-8B (FC) + Toollery	93.50	98.00 $\uparrow$	90.50	99.00 $\uparrow$	91.50 $\uparrow$	88.50 $\uparrow$	85.88 $\uparrow$
Qwen3-235B (FC)	98.00	93.00	87.50	89.50	85.50	80.00	81.13
Qwen3-235B (FC) + Toollery	98.50 $\uparrow$	92.50	89.00 $\uparrow$	89.50	90.00	91.50 $\uparrow$	88.63 $\uparrow$

Table 9: BFCL-V4 non-live multiple results

Model	≤ 10	20	30	40	50	100	443
Qwen3-4B (Prompt)	95.50	99.50	97.50	96.50	95.50	90.00	24.25
Qwen3-4B (Prompt) + Toollery	100.00 $\uparrow$	97.50	98.00 $\uparrow$	95.00	98.00 $\uparrow$	91.50 $\uparrow$	83.63 $\uparrow$
Qwen3-8B (Prompt)	97.50	98.50	99.50	97.50	97.00	89.50	37.63
Qwen3-8B (Prompt) + Toollery	99.50 $\uparrow$	100.00 $\uparrow$	97.50	96.00	98.00 $\uparrow$	91.00 $\uparrow$	80.63 $\uparrow$
DeepSeek V3.2 (Prompt)	86.00	99.50	97.00	94.50	96.50	85.50	70.25
DeepSeek V3.2 (Prompt) + Toollery	97.50 $\uparrow$	97.50	95.00	98.00 $\uparrow$	98.50 $\uparrow$	89.50 $\uparrow$	81.38 $\uparrow$
GPT-5-mini (Prompt)	88.00	87.50	92.50	85.00	88.50	88.00	80.00
GPT-5-mini (Prompt) + Toollery	88.00	89.50 $\uparrow$	91.50	86.50 $\uparrow$	95.00 $\uparrow$	90.00 $\uparrow$	82.00 $\uparrow$
GPT-5-mini (FC)	79.50	69.50	65.50	72.00	70.50	63.50	58.63
GPT-5-mini (FC) + Toollery	81.50 $\uparrow$	77.00 $\uparrow$	75.50 $\uparrow$	73.00 $\uparrow$	76.00 $\uparrow$	69.50 $\uparrow$	68.63 $\uparrow$
Qwen3-8B (FC)	97.50	97.50	97.50	98.50	96.00	85.00	
Qwen3-8B (FC) + Toollery	99.50 $\uparrow$	96.50	97.50	96.50	97.00 $\uparrow$	90.50 $\uparrow$	86.75 $\uparrow$
Qwen3-235B (FC)	95.50	99.50	96.50	96.00	97.50	80.00	79.13
Qwen3-235B (FC) + Toollery	96.00 $\uparrow$	99.00	98.50 $\uparrow$	98.00 $\uparrow$	98.00 $\uparrow$	93.50 $\uparrow$	94.00 $\uparrow$

candidate set before final LLM decision-making, rather than asking the final LLM to select from the full library.

C Failure Case Analysis

Table 10 shows representative retrieval failures from SkillsBench and BFCL-live. These cases suggest that many Toollery failures are near misses rather than unrelated retrieval errors. In SkillsBench, failures often reflect two intertwined issues: incomplete coverage of multi-capability workflows and mismatch between the benchmark’s exact gold capability set and functionally similar retrieved capabilities. In BFCL-live, failures more often arise from fine-grained ambiguity among atomic tool interfaces, where the retrieved tools belong to the right tool family but do not match the exact gold interface.

Functionally similar capabilities can fail exact-gold matching. Several failures in Table 10 are near misses under exact-label evaluation. In the SkillsBench build-repair case, the benchmark gold capabilities are Maven-specific, including `gt/maven-build-lifecycle`, `gt/maven-dependency-management`, and `gt/maven-plugin-configuration`. Toollery instead retrieves generic CI and debugging capabilities such as `gt/analyze-ci`, `gt/uv-package-manager`, and `development/ci-debug`, which are plausible for repairing build failures but do not match the exact gold labels. BFCL-live shows the same issue at the atomic-tool level: for a request asking for help with `list_files`, Toollery retrieves `list_files`, `list_directory_contents`, and generic help tools, but misses the annotated gold tool `dartfx_help`. For `flight-price` and `news-`

Dataset	Query / task	Gold capability	Top-5 retrieval
SkillsBench	You need to fix build errors in a Java codebase. The repository is located in ...	gt/maven-build-lifecycle gt/maven-dependency-management gt/maven-plugin-configuration	gt/analyze-ci gt/uv-package-manager development/ci-debug other/creating-commit other/github-pr-fix
SkillsBench	Build an itinerary for the user according to the following requirements: We require a 7-day travel itinerary for two leaving from Minneapolis ...	gt/search-accommodations gt/search-attractions gt/search-cities gt/search-driving-distance gt/search-flights gt/search-restaurants	other/travel-planner-ai-claude-skills other/tour-operations gt/search-accommodations development/travel-planner other/travel-requirements-expert
SkillsBench	In this task, you are given a clip of screen recording of a player playing the game Super Mario. Your goal is to analyze the key frames in this video and count how many coins / enemies / turtles ever showing up in each frame and write the results into a CSV file.	gt/ffmpeg gt/image_editing gt/object_counter	gt/object_counter gt/ffmpeg gt/videoframeextraction other/videoprocessing data/videoframereader
BFCL-live	I'm having trouble understanding how to use the list_files functionality. Help me.	dartfx_help	list_files list_directory_contents help list_projects adriel_list_projects
BFCL-live	search flight prices from NYC to NEW Delhi use tools, finally respond in json	search_web_tool	get_trip Flights_4_SearchRoundtripFlights Flights_4_SearchOnewayFlight render_ui_finish Trains_1_GetTrainTickets
BFCL-live	Find a recent news article about corona virus	search_web_tool	search_engine.query get_news_report HNA_WQA.search HNA_WQA.query HNA_NEWS.search

Table 10: Representative Toollery failure cases. Skill-routing failures often miss implementation-specific or complementary capabilities, while BFCL-live failures often arise from near-duplicate tool interfaces or underspecified requests.

search requests, Toollery retrieves flight-specific, news-specific, or search-like tools, but misses the gold `search_web_tool`. These cases suggest that some retrieval failures reflect exact-interface or exact-label mismatch rather than completely unrelated retrieval.

Broad capabilities can substitute for decomposed gold labels. The travel-planning case illustrates a capability-granularity mismatch. The user request asks for an end-to-end itinerary, and Toollery retrieves broad travel-planning capabilities such as travel-planner-style agents, along with one exact gold capability, `gt/search-accommodations`. The benchmark gold set, however, decomposes the task into several fine-grained capabilities, including attraction search, driving-distance search, flight search, and

restaurant search. Under exact recall, the broad travel-planner capability receives no credit for covering these subfunctions, even if it may be able to implement a complete itinerary workflow. A similar pattern appears in the Super Mario video-analysis case: Toollery retrieves broad video-processing or task-level capabilities together with `gt/object_counter` and `gt/ffmpeg`, but misses the exact gold `gt/image_editing`. These cases show that some failures reflect mismatch between the granularity of retrieved capabilities and the granularity of benchmark annotations.

Overall, the failure cases show two main boundary conditions for Toollery. First, in large capability libraries, multiple capabilities may be functionally overlapping, so exact-label evaluation can count feasible alternatives as failures. Second, for compositional tasks, a broad retrieved capa-

bility may cover an end-to-end workflow while the benchmark gold set decomposes the task into finer-grained capabilities. Future improvements should therefore consider interface-aware reranking for near-duplicate tools and staged retrieval for multi-capability workflows.